Cloud Computing Mini Project - Report

Name: Lokesh Yadav
Roll number: 230101039
Repository: lokesh1-cell/cloud-assignmnet

Evidence status (8 November 2026). This report separates the supplied historical outputs in `results/` from checks rerun during the submission audit. The saved model scores were recomputed successfully, but model weights, raw teacher shards, the cleaned training dataset, and a link identifying the original cluster run were not available. Historical logs are evidence supplied with the project, not proof that this repository has reproduced those runs. The submission remains incomplete.

Architecture overview

The intended system uses a local k3d cluster with SeaweedFS in namespace `data`, teacher workers in `pipeline`, and the inference service in `app`. Ollama serves `phi3.5`; workers label commands, cleaning produces the student training set, and CPU training fine-tunes `EleutherAI/pythia-70m`. Model files belong in object storage and GitHub Release assets, outside Git and the service image. FastAPI loads the model at startup and exposes the parser through a Kubernetes Service and Ingress.

Task 1 - Object storage and parallel teacher labelling

Design and implementation. `cluster/k3d.yaml` declares one server and one agent. SeaweedFS has persistent storage and an S3 service; `scripts/init_storage.py` creates `datasets` and `models` and uploads the provided command pool. The indexed Job has two completions and parallelism two. Workers use zero-based line-number modulo `SHARD_COUNT` to select a shard; `JOB_COMPLETION_INDEX` supplies the shard index when `SHARD_INDEX` is absent. The prompt requests the four required keys; the default teacher settings are temperature zero and seed 42.

Cleaning rejects invalid JSON, wrong keys, unsupported intents, invented slot values, and duplicate command texts, keeping the smallest ID and sorting by ID. It writes the cleaned JSONL and SHA-256. The supplied counts are:

Input	Invalid JSON	Wrong keys	Bad intent	Bad slot	Duplicates	Kept
3000	0	1	3	86	49	2861

Verified: the partition adds to 3000 and cleaning unit tests pass. The command pool matches the starter ZIP byte for byte. **Not verified:** execution of two workers in `pipeline`, exactly-once shard coverage, bucket contents, or the cleaned file checksum, because the shard and training artifacts are missing. The existing worker strips outer whitespace from the response string; exact raw-reply preservation requires review. Recover the original shards or regenerate them without manual editing. A queued worker design could reduce shard imbalance.

Task 2 - Student training and evaluation

Design and configuration. Training is sequence-level distillation: the target is the teacher's generated JSON, conditioned on `Command: <text>` followed by `JSON:`. Prompt tokens are masked in the loss. The configuration records seed 42, learning rate 0.00005, batch size 8, three epochs, maximum length 128, and a 1500-example limit. Version two uses seed 7. CPU execution lowers hardware requirements but limits throughput. `make train SUBSET=50` provides a smoke-run interface.

Recomputed results: the provided scorer reproduces every saved score from the saved prediction files on the unchanged 100-example validation set:

Model	JSON validity	Intent	Date	Time	Place	Full frame
Teacher	100%	85%	80%	88%	85%	48%
Untrained base	0%	0%	0%	0%	0%	0%
Student v1	99%	82%	77%	85%	82%	43%
Student v2	97%	78%	71%	88%	85%	44%

These checks establish internal consistency of the saved predictions and reports; they do not establish which unavailable weights generated the predictions. The v1 full-frame score is above half the teacher score on this validation set; hidden set performance is unknown. Supplied v1 epoch losses are 0.257248, 0.104621, and 0.072164. `training/loss.csv` matches the supplied v2 loss record, not v1.

Sequence-level distillation learns hard text targets. Logit-based distillation instead learns from per-token probability distributions. This implementation's Ollama generation interface supplies text, not the distributions needed by the training procedure; the two model tokenizers also differ. The 70M student is small and CPU-friendly but makes slot and intent errors. Cleaner teacher labels and additional experiments within the required training cap could improve it.

Outstanding: restore or retrain both model versions, verify a 50-example smoke run, store the models and tokenizers in object storage, publish release archives and checksums, and regenerate predictions from those exact weights. No trained model release was present in the target repository at the audit.

Task 3 - Inference service in a container

Implementation. FastAPI listens on port 8000. `/healthz` reports liveness; `/readyz` stays 503 until model loading completes. `/parse` returns the four-key object and model-version/pod headers, returns 503 while unready, and returns {`"error":"invalid_model_output"`} with HTTP 500 for schema-invalid output. `FAULT_INJECT=1` exercises that error path. The loader uses `MODEL_URI`, `MODEL_VERSION`, `S3_ENDPOINT`, and `MAX_NEW_TOKENS`; Kubernetes supplies storage credentials from Secrets. The Dockerfile installs CPU PyTorch and copies source and schema without model files. Declared runtime packages are version-pinned; transitive packages are not captured in a complete lock file.

Verified: API unit tests pass using a stubbed model, including readiness, headers, valid output, invalid output, and fault injection. **Supplied evidence:** local inference/endpoints logs and an image-size record of 1.74 GB. These were not rerun with actual weights. A `kubectl top pods` snapshot records 452 MiB and 475 MiB, which is not a continuous peak-memory measurement.

Outstanding: build and run the actual image, issue 50 sequential requests at 1 CPU and 1 GiB, record exit/OOM status and peak memory. The supplied 10-command local inference log and cluster snapshot do not meet that specific test. Improve performance through measured batching; verify unusual malformed output types too.

Task 4 - Deployment, scaling, and reliability

The Deployment specifies two replicas, requests of 250m CPU/512 MiB, limits of 1 CPU/1 GiB, liveness/readiness probes, and a rolling-update strategy with zero unavailable replicas and one surge replica. Service and Ingress expose `/parse`. The HPA targets 70% CPU with one to two replicas. Supplied evidence shows:

Check	Recorded evidence	Assessment
Rollout	2 failures / 39 requests = 5.13%	Exceeds required 0.5%; rollback header evidence absent
Recovery	Two ready pods after one replacement	Recovery load-test error rate not recorded
Cold start	8 s, 8 s, 8 s; median 8 s	Timer origin and explicit S3 download evidence insufficient
Balancing	109 and 104 successful responses	About 51.2% and 48.8%; both exceed 30%
HPA	196% utilization; scaling limited at two	Does not prove a 1-to-2-to-1 scaling cycle

A rollout configuration cannot establish that the required failure-rate target was met. The failed requests still count; their precise cause is not established. Startup logs show `model loaded` but do not identify an S3 download. Repeat these experiments with timestamped logs, explicit storage-download logging, headers after rollback, and simultaneous load/recovery counters. Confirm that no Service endpoint is ready while the model is loading. Both k3d nodes run on the same host, so the setup cannot demonstrate fault tolerance across physical machines.

Task 5 - Subnets and stateless access control

Allocate the largest requirement first within `10.20.0.0/16`. A prefix must satisfy `2^(32-prefix)-2 >= hosts`; the resulting minimum blocks are contiguous:

Subnet	Network/prefix	Usable range	Broadcast	Usable hosts
app	10.20.0.0/22	10.20.0.1 - 10.20.3.254	10.20.3.255	1022
public	10.20.4.0/24	10.20.4.1 - 10.20.4.254	10.20.4.255	254
data	10.20.5.0/25	10.20.5.1 - 10.20.5.126	10.20.5.127	126
management	10.20.5.128/27	10.20.5.129 - 10.20.5.158	10.20.5.159	30

The public-subnet NACL blocks the abusive range before inbound allows, permits HTTP/HTTPS globally, SSH only from management, and TCP ephemeral replies from app. Outbound allows TCP 8000 to app and TCP ephemeral replies to clients. All other traffic is denied. NACLs are stateless, so replies need separate rules. First-match ordering means the abusive-range deny must have a lower rule number than web allows. **Verified:** all six sample packets and subnet tests pass. Hidden packet results remain unknown. These are conceptual network rules, not enforcement of these CIDRs by the local k3d cluster. Stateful per-service controls would be finer.

Task 6 - Reproducibility, tests, and CI/CD

The audit added the missing GitHub Actions workflow and credential/model ignore rules. CI runs lint, unit tests, release-model download with SHA-256 verification, model evaluation, and a 95% JSON-validity gate. A separate job runs Gitleaks on Git history. A manual `untrained` option evaluates the actual base model to demonstrate a failing quality gate. Model files are expected as `v1.tar.gz` and `model.sha256` on release `model-v1`; the latter contains the archive checksum. A v2 archive must also be published for the reliability experiments.

Verified locally: all 22 existing tests, lint, shell syntax, kit-file integrity, saved-score reproduction, and network checks. The application Python logic is unchanged by the formatting and documentation revision. Training and deployment were not rerun because the required artifacts and local Docker/k3d/Ollama tools were absent. `make up` and `make down` exist, but a fresh-clone deployment cannot be claimed successful without the model release and dependencies.

Verified CI: the deliberate bad-model run ran the untrained base on all 100 validation examples and failed specifically at the quality gate: JSON validity 0%, below 95%. Lint, all 22 tests, model inference, and the secret scan passed in that run. Generated predictions and scores are saved under `results/submission_audit/ci_bad_model/`. The normal release run is blocked by the absent trained-model release. This is distinct from the intentional quality-gate failure. `AI_USE.md` records this audit's assistance.

Task 7 - Performance, cost, and review

The saved cluster run used four users for **90 seconds**, so it is not the required two-minute standard run. Its final console summary records 215 requests, two HTTP 500 failures, p50 1600 ms, p95 3300 ms, and 2.41 requests/s. The saved CSV records 213 requests, two failures, and 2.4024 requests/s, while successful-response headers total 213. The files therefore reflect different counting snapshots and should not be presented as one reconciled dataset. Repeat the standard command for 2m.

For a clearly identified historical console snapshot, success is 213/215 = 99.07% and error rate is 0.93%. A 99% success SLO over that 90-second run gives an error budget of 2.15 requests: two whole failures allowed, both used (93.02% of the fractional budget). The CSV snapshot independently gives 211/213 = 99.06% and an error budget of 2.13 requests. Neither snapshot is a fresh measured run.

Indicative cost using the console's rounded throughput: assume two replicas billed at limits of 1 vCPU and 1 GiB. Per replica per hour = 1 x 3.50 + 1 x 0.45 = 3.95 INR. Total = 7.90 INR/hour. Requests/hour = 2.41 x 3600 = 8676. Cost per 1000 requests = 7.90 / 8676 x 1000 = **0.9106 INR**. Using the CSV rate instead gives about 0.9134 INR. The rate card is hypothetical; these are estimates. The saved resource snapshot is 953m CPU/452 MiB and 906m CPU/475 MiB; cold-start median from the supplied timings is 8 s, subject to the Task 4 evidence limits.

Concern	Recorded or rerun evidence	Limitation	Improvement
Data/storage	2861 kept in supplied counts	Raw shards missing	Restore shards and verify checksum
Model quality	Recomputed v1 full frame 43%	Weights unavailable	Publish exact versioned models
Latency/throughput	Saved p95 3300 ms; 2.41 rps	90 s run and inconsistent totals	Rerun standard 2m test
Resources	Snapshot 452/475 MiB	Not a measured memory peak	Sample during 50 sequential requests
Reliability/recovery	Saved rollout 2/39 failures	Above 0.5%; recovery error rate absent	Repeat under measured load
Security/cost	NACL 6/6; estimated 0.9106 INR/1000	Broad S3 identity and assumed billing	Scope credentials and rerun cost

Submission status and references

The report and source are prepared, but missing model/data artifacts and live-run evidence prevent a complete compliance claim. The student will supply the handwritten reflection as `230101039.pdf` (maximum three pages). Package the report, reflection, sources, results, and README into `230101039.zip`, and tag the complete submission commit `final` only after outstanding requirements are resolved.

References: the supplied CS3081 assignment and starter kit; Bastianelli et al., SLURP (EMNLP 2020, CC BY 4.0); Kim and Rush, Sequence-Level Knowledge Distillation (EMNLP 2016); project configuration and evidence files referenced above.